

Week 3 Module:

Date: 29/06/2026

Question 1 (Easy):

A university's Training and Placement Cell have successfully completed its annual placement drive. The placement department has collected the salary packages (in LPA) offered to students by different companies. However, the records were entered in the order they were received, resulting in an unsorted list of package values.

Before publishing the placement statistics report, the department wants all package values arranged in ascending order. Since the university plans to handle placement data for thousands of students in the coming years, they want a sorting technique that is efficient for large datasets.

Your task is to implement the **Merge Sort algorithm** to sort the package values in increasing order. Merge Sort follows a divide-and-conquer approach by dividing the dataset into smaller subarrays, sorting them recursively, and then merging them back together.

The final sorted list will help the university identify minimum, median, and maximum packages while generating analytical reports.

Input

- First line contains an integer **N**, representing the number of placed students.
- Second line contains **N space-separated integers**, representing package values (multiplied by 100 to avoid decimals).

Output

Print the package values in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $100 \leq \text{Package} \leq 10000$

Example

Input:

6
450 1200 800 650 300 1500

Output:

300 450 650 800 1200 1500

Question 2 (Medium):

A large e-commerce company tracks the time taken (in minutes) to process customer orders before shipment. Due to varying warehouse workloads, processing times differ significantly from one order to another.

The analytics team wants to study operational efficiency by sorting all processing times and generating statistical information. Since millions of orders are processed every month, the company requires an efficient sorting technique with predictable performance.

Your task is to use **Merge Sort** to arrange all processing times in ascending order.

After sorting, perform the following operations:

1. Display the sorted processing times.
2. Find the **median processing time**.
3. Count the number of orders whose processing time is greater than the median.
4. Display the difference between the fastest and slowest processing time.

The solution should efficiently handle large datasets and demonstrate a clear understanding of Merge Sort's divide-and-conquer strategy.

Input

- First line contains integer **N**.
- Second line contains **N space-separated processing times**.

Output

- Sorted processing times.
- Median processing time.
- Count of orders greater than median.
- Difference between maximum and minimum processing time.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{Time} \leq 100000$

Example

Input:

7
12 25 18 30 15 22 40

Output:

12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28

Topic 2: Quick Sort

Question 1 (Easy):

A cloud infrastructure company manages hundreds of servers distributed across different geographical locations. Every hour, each server reports its average response time in milliseconds. The collected data is stored in the order it arrives and is not sorted.

To generate performance reports, the operations team wants the response times arranged from the fastest server to the slowest server. Since the data size can become very large, an efficient in-place sorting technique is required.

Your task is to implement **Quick Sort** to sort the response times in ascending order. Built-in sorting methods are not allowed.

After sorting, the operations team will use the report to identify high-latency servers and optimize system performance.

Input

- First line contains integer **N**.
- Second line contains **N space-separated response times**.

Output

Print the response times in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{Response Time} \leq 10000$

Example

Input:

6
120 45 78 200 60 95

Output:

45 60 78 95 120 200

Question 2 (Medium):

A financial analytics company receives daily trade-value data from multiple stock exchanges. Analysts use this information to identify high-value trading activity and market trends.

The trade values are received in random order throughout the day. Before analysis can begin, the data must be sorted in descending order.

Your task is to implement **Quick Sort** to arrange the trade values from highest to lowest.

After sorting, perform the following operations:

1. Display the sorted trade values.
2. Find the **Top 5 highest trade values**.
3. Calculate the average of the Top 5 values.
4. Count how many trade values are greater than the overall average of all trade values.

This problem evaluates your understanding of partitioning, recursion, and real-world data analysis using Quick Sort.

Input

- First line contains integer **N**.
- Second line contains **N space-separated trade values**.

Output

- Sorted trade values (descending).
- Top 5 values.
- Average of Top 5 values.
- Count of values greater than overall average.

Constraints

- $5 \leq N \leq 100000$
- $1 \leq \text{Trade Value} \leq 10^9$

Example

Input:

8
500 1200 700 2000 900 1500 3000 2500

Output:

3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 4

Topic 3: Heap Sort

Question 1 (Easy):

An online gaming platform is hosting a national-level tournament involving thousands of participants. At the end of the tournament, player scores are collected from multiple game servers and stored in random order.

Before announcing the final rankings, the organizers need all scores sorted in ascending order. Since the leaderboard data can be very large, they have decided to use **Heap Sort**, which provides guaranteed $O(N \log N)$ performance.

Your task is to implement Heap Sort and arrange all player scores in increasing order.

The sorted output will help tournament officials generate rankings and determine qualification thresholds for future events.

Input

- First line contains integer **N**.
- Second line contains **N space-separated player scores**.

Output

Print the scores in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $0 \leq \text{Score} \leq 1000000$

Example

Input:

6
850 1200 950 600 1500 1000

Output:

600 850 950 1000 1200 1500

Question 2 (Medium):

A multi-specialty hospital maintains records of emergency response times for all critical cases handled during a month. Each response time represents the number of minutes taken by the emergency team to reach and begin treatment for a patient.

Hospital administrators want to analyze the efficiency of their emergency services. To do so, all response times must first be sorted using **Heap Sort**.

After sorting the response times in ascending order, generate the following statistics:

1. Display the sorted response times.
2. Find the fastest response time.
3. Find the slowest response time.
4. Calculate the average response time.
5. Count the number of cases handled faster than the average.
6. Calculate the percentage of cases handled faster than the average.

The hospital intends to use this information to improve resource allocation and reduce emergency response delays.

Input

- First line contains integer **N**.
- Second line contains **N space-separated response times**.

Output

- Sorted response times.
- Fastest response time.
- Slowest response time.
- Average response time.
- Number of cases faster than average.
- Percentage of cases faster than average.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{Time} \leq 500$

Example

Input:

8
12 7 15 9 20 5 10 8

Output:

5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%

```

#include <bits/stdc++.h>
using namespace std;

void merge(vector<int> &arr, int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(vector<int> &arr, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(arr, l, m);
    mergeSort(arr, m + 1, r);
    merge(arr, l, m, r);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    vector<int> pkg(n);
    for (int i = 0; i < n; i++) cin >> pkg[i];

    mergeSort(pkg, 0, n - 1);

    for (int i = 0; i < n; i++)
        cout << pkg[i] << (i == n - 1 ? '\n' : ' ');

    return 0;
}

```

Console – Output

```

$ 6
$ 450 1200 800 650 300 1500
300 450 650 800 1200 1500

```

Sample Input

```

6
450 1200 800 650 300 1500

```

Expected Output

```

300 450 650 800 1200 1500

```

```

#include <bits/stdc++.h>
using namespace std;

void merge(vector<int> &arr, int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(vector<int> &arr, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(arr, l, m);
    mergeSort(arr, m + 1, r);
    merge(arr, l, m, r);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    vector<int> t(n);
    for (int i = 0; i < n; i++) cin >> t[i];

    mergeSort(t, 0, n - 1);

    for (int i = 0; i < n; i++)
        cout << t[i] << (i == n - 1 ? '\n' : ' ');

    double median;
    if (n % 2 == 1) median = t[n / 2];
    else median = (t[n / 2 - 1] + t[n / 2]) / 2.0;

    int aboveMedian = 0;
    for (int i = 0; i < n; i++)
        if (t[i] > median) aboveMedian++;

    int diff = t[n - 1] - t[0];

    if (median == (long long)median)
        cout << "Median: " << (long long)median << '\n';
    else
        cout << "Median: " << median << '\n';

    cout << "Orders Above Median: " << aboveMedian << '\n';
    cout << "Difference: " << diff << '\n';

    return 0;
}

```

Console – Output

```

$ 7
$ 12 25 18 30 15 22 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28

```

Sample Input

```

7
12 25 18 30 15 22 40

```

Expected Output

```

12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28

```



```

#include <bits/stdc++.h>
using namespace std;

int partition(vector<int> &arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(vector<int> &arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    vector<int> resp(n);
    for (int i = 0; i < n; i++) cin >> resp[i];

    quickSort(resp, 0, n - 1);

    for (int i = 0; i < n; i++)
        cout << resp[i] << (i == n - 1 ? '\n' : ' ');

    return 0;
}

```

Console — Output

```

$ 6
$ 120 45 78 200 60 95
45 60 78 95 120 200

```

Sample Input

```

6
120 45 78 200 60 95

```

Expected Output

```

45 60 78 95 120 200

```

```

#include <bits/stdc++.h>
using namespace std;

int partition(vector<long long> &arr, int low, int high) {
    long long pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] >= pivot) { // descending order
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(vector<long long> &arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    vector<long long> trade(n);
    long long total = 0;
    for (int i = 0; i < n; i++) {
        cin >> trade[i];
        total += trade[i];
    }

    quickSort(trade, 0, n - 1);

    for (int i = 0; i < n; i++)
        cout << trade[i] << (i == n - 1 ? '\n' : ' ');

    cout << "Top 5:";
    long long top5sum = 0;
    int k = min(5, n);
    for (int i = 0; i < k; i++) {
        cout << ' ' << trade[i];
        top5sum += trade[i];
    }
    cout << '\n';

    double top5avg = (double)top5sum / k;
    if (top5avg == (long long)top5avg)
        cout << "Average of Top 5: " << (long long)top5avg << '\n';
    else
        cout << fixed << setprecision(2) << "Average of Top 5: " << top5avg << '\n';

    double overallAvg = (double)total / n;
    int countAbove = 0;
    for (int i = 0; i < n; i++)
        if (trade[i] > overallAvg) countAbove++;

    cout << "Values Above Overall Average: " << countAbove << '\n';

    return 0;
}

```

Console – Output

```

$ 8
$ 500 1200 700 2000 900 1500 3000 2500
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3

```

Sample Input

```

8
500 1200 700 2000 900 1500 3000 2500

```

Expected Output

```

3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3

```

```

#include <bits/stdc++.h>
using namespace std;

void heapify(vector<int> &arr, int n, int i) {
    int largest = i;
    int l = 2 * i + 1, r = 2 * i + 2;

    if (l < n && arr[l] > arr[largest]) largest = l;
    if (r < n && arr[r] > arr[largest]) largest = r;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(vector<int> &arr) {
    int n = arr.size();
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    vector<int> score(n);
    for (int i = 0; i < n; i++) cin >> score[i];

    heapSort(score);

    for (int i = 0; i < n; i++)
        cout << score[i] << (i == n - 1 ? '\n' : ' ');

    return 0;
}

```

Console - Output

```

$ 6
$ 850 1200 950 600 1500 1000
600 850 950 1000 1200 1500

```

Sample Input

```

6
850 1200 950 600 1500 1000

```

Expected Output

```

600 850 950 1000 1200 1500

```

```

#include <bits/stdc++.h>
using namespace std;

void heapify(vector<int> &arr, int n, int i) {
    int largest = i;
    int l = 2 * i - 1, r = 2 * i + 1;

    if (l < n && arr[l] > arr[largest]) largest = l;
    if (r < n && arr[r] > arr[largest]) largest = r;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(vector<int> &arr) {
    int n = arr.size();
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    vector<int> t(n);
    long long total = 0;
    for (int i = 0; i < n; i++) {
        cin >> t[i];
        total += t[i];
    }

    heapSort(t);

    for (int i = 0; i < n; i++)
        cout << t[i] << (i == n - 1 ? '\n' : ' ');

    int fastest = t[0], slowest = t[n - 1];
    double avg = (double)total / n;

    int fasterCount = 0;
    for (int i = 0; i < n; i++)
        if (t[i] < avg) fasterCount++;

    double pct = (double)fasterCount / n * 100.0;

    cout << "Fastest: " << fastest << '\n';
    cout << "Slowest: " << slowest << '\n';
    cout << fixed << setprecision(2) << "Average: " << avg << '\n';
    cout << "Cases Faster Than Average: " << fasterCount << '\n';
    cout << "Percentage: " << pct << "%\n";

    return 0;
}

```

Console - Output

```

$ 8
$ 12 7 15 9 20 5 10 8
$ 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%

```

Sample Input

```

8
12 7 15 9 20 5 10 8

```

Expected Output

```

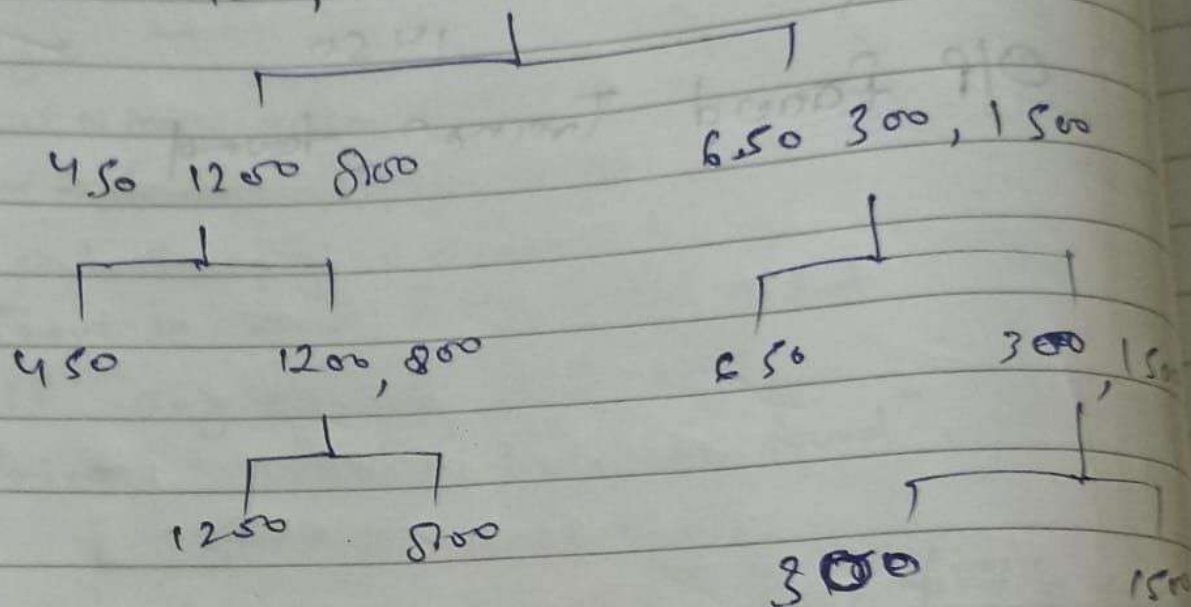
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%

```


Q1 N package random
Sort → merge Sort.

Ex 450, 1200, 800, 650, 300, 1500

Q1-1
Divide



Q1-2 merge Sort

• merge

left	merge		
Half	1200 > 800	450, 800, 1200	
	800 first	450 < 800	
	→ 800, 1200	450	<u>450, 800</u>

Right half	300, 1500	650, 300, 1500
	300 < 1500	650 > 300
	<u>300, 1500</u>	650 < 1500
		<u>300, 650, 1500</u>

Left
450, 800, 1200

Right
300, 650, 1500

L	R	Smaller	O/P
450	300	300	300
450	650	450	300, 450
800	650	650	300, 450, 650
800	1500	800	300, 450, 650, 800
1200	1500	1200	300, 450, 650, 800, 1200
	1500	1500	[300, 450, 650, 800, 1200, 1500]

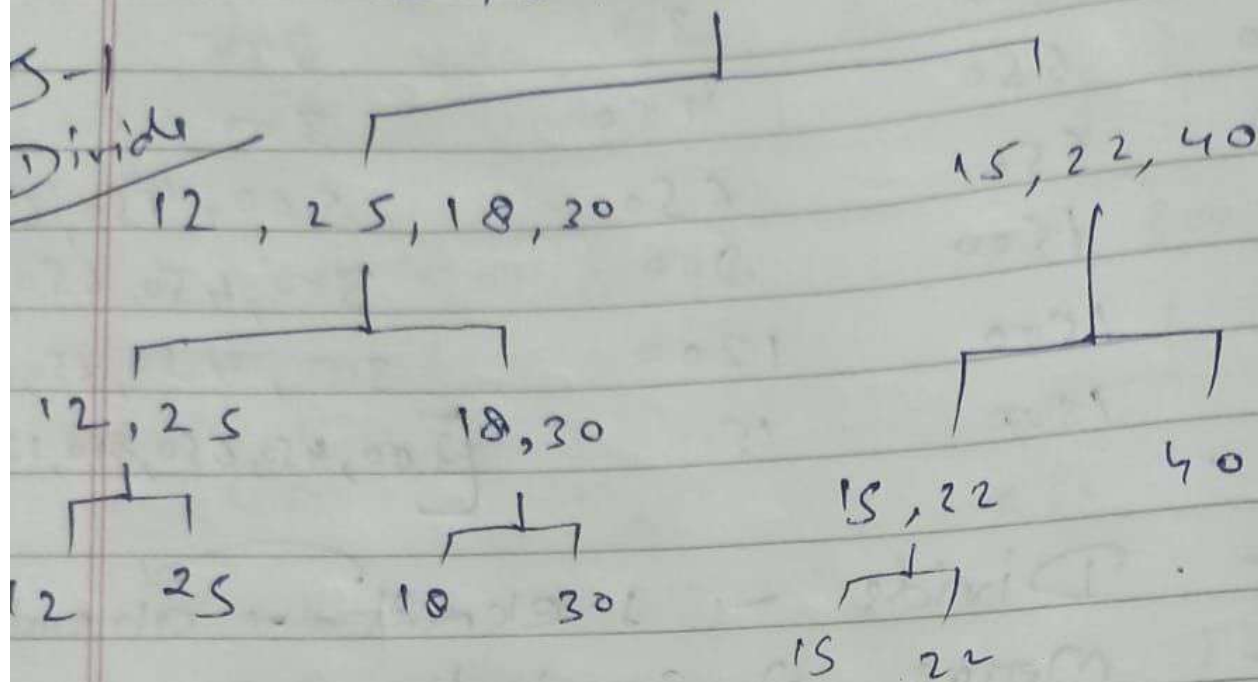
~~##~~ Divide $\rightarrow$ 1 element $\rightarrow$ already sorted
~~##~~ merge 2 sorted arr into one larger sorted arr.

~~##~~ Keep Merging until whole arr become sorted.

Merge sort = Divide $\downarrow \downarrow \downarrow$
 then merge $\uparrow \uparrow \uparrow$

Q 2 merge Sort + Median + Count + Difference

Ex 12, 25, 18, 30, 15, 22, 40



3-2

merge

12, 25

12 < 18

25 > 18

12 < 30

25 < 30

18, 30

10, 30

12, 18, 25, 30

left

Right

15, 22, 40

15 < 40

22 < 40

15, 22, 40

12 < 15

18 > 15

25 > 22

30 < 40

12	15	18	22	25	30	40
0	1	2	3	4	5	6

median $\rightarrow$ 7 Element

$$n = 7 \rightarrow n / 2 = 3$$

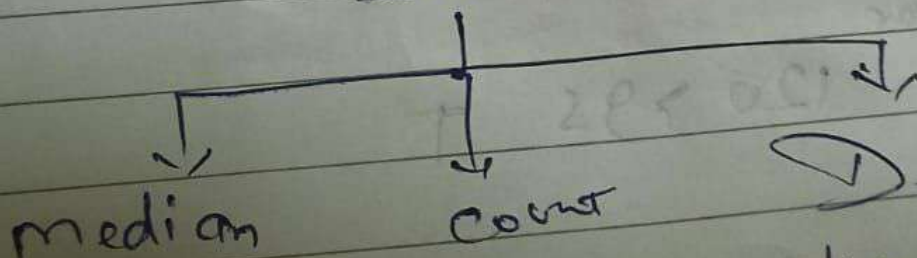
$$\text{Median} = \text{arr}[3] = 22$$

Count $\rightarrow$ element greater than 22.
25, 30, 40
count = 3

Difference Target - Smallest
 $40 - 12 = 28$

Approach $\rightarrow$ Take Arr :

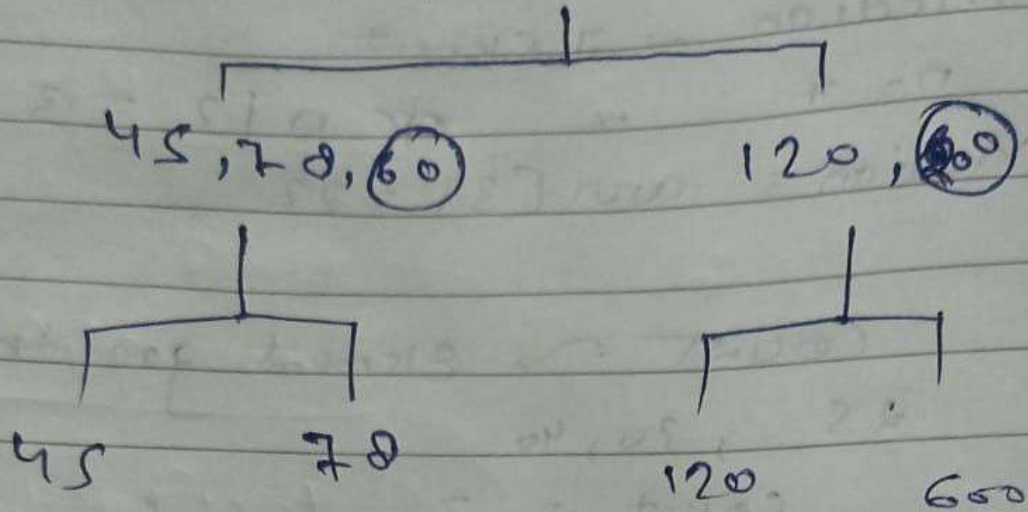
merge Sort
 $\rightarrow$ Sorted Arr



median (max - min)

Topic 2: Quick Sort

Q1 120, 45, 70, 200, 60, 95 ^{first}



Q-1



scan every element

keep track of posn

where next smaller element should go

$$i = -1 \quad | \quad j = 0$$

Q-2 Compare

$$i = -1$$

$$j = 0$$

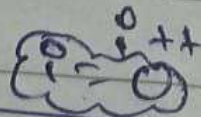
$$120 > 95 \quad T$$

$$i = -1$$

$$j = 1$$

$$45 < 95$$

~ Swap



arr[i] and
arr[j]

45, 120, 70, 200, 60, 95

i

j

• $70 < 95$ $\swarrow$ swap
 $i++ \Rightarrow i = 1$

45, 70, 120, 200, 60, 95
 $\uparrow$ $\downarrow$

• 200
 $200 > 95$ Nothing

60
 $60 < 95$ swap
 $i = 2$

45, 70, 60, 200, 120, 95
 $\uparrow$

• Swap arr[i+1] with pivot
45, 70, 60, 95, 120, 200
 $\uparrow$
 pivot fix

45, 70, 60 < 95

95 < 120, 200

Quick Sort

Quick
Sort

* left Recursion 45, 70, (60) pivot
 partition 45 < 60 ✓

Swap pivot 45, 60, 70
 70 > 60 ✗

classmate
Date _____
Page _____

* Right Recursion 120 200

$120 < 200 \quad \checkmark$

45, 60, 78, 95, 120, 200

Approach

Q5-1 Choose Pivot

Q5-2 Place pivot in its correct posn
by partition

S-3 Element smaller than pivot $\rightarrow$ left

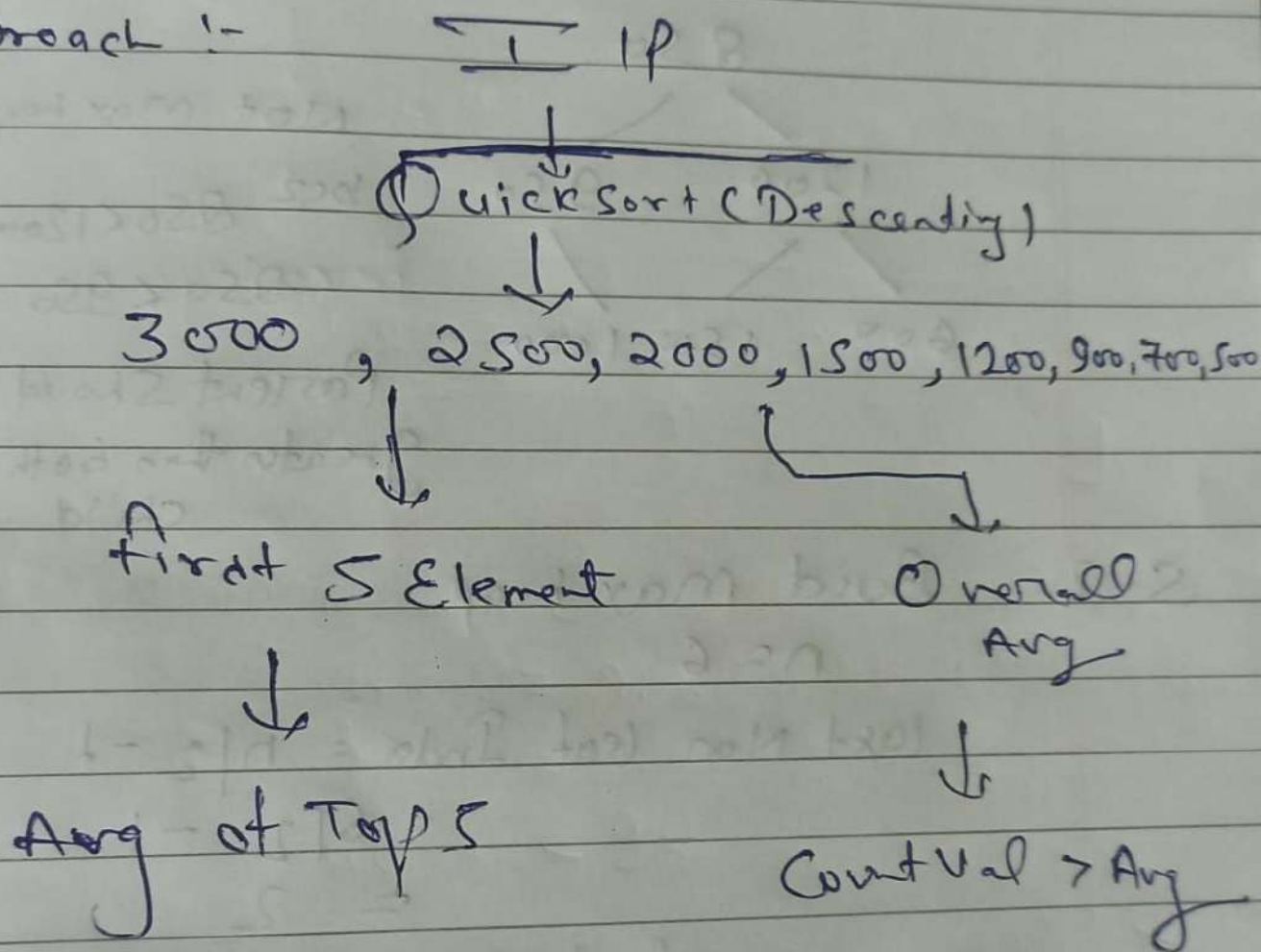
Q5-4 Element greater than pivot $\rightarrow$ right

S-5 Recursively repeat same process

Q2 Problem:

- ① Sort in desc Order $\rightarrow$ Quick Sort
- ② Print Top 5 highest val
- ③ find Avg of Top 5 val
- ④ count how many val are greater than overall avg.

Approach :-



Topic 3: Heap Sort

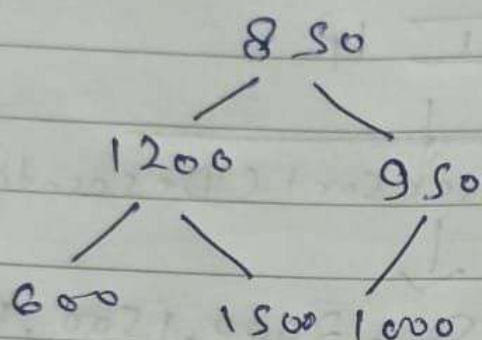
Q1

Ex 850, 1200, 950, 600, 1500, 1000
 0 1 2 3 4 5

Q1 - Convert Arr into Binary tree

Left Child = $2 \times i + 1$

Right = $2 \times i + 2$



Not Max heap

bcz $850 < 1200$

$850 < 950$

Parent should
greater than both
child

Q2 - Build max heap
 $n = 6$

last Non leaf Index = $n/2 - 1$

= $6/2 - 1$

= 2

So heapify index

$2 + 1 \rightarrow 0$

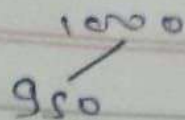
* Heapify Index 2

Node = ~~950~~ 950

Child: left = 1000

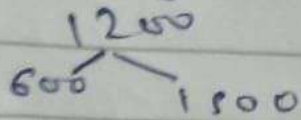
Right = None

1000 > 950 swap

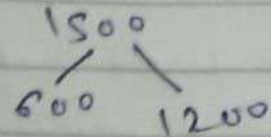


→ 850, 1200, 1000, 600, 1500, 950

* Heapify index = 1

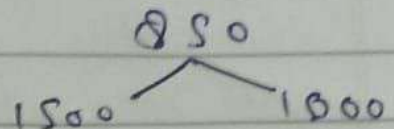
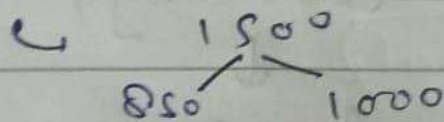


1200 < 1500

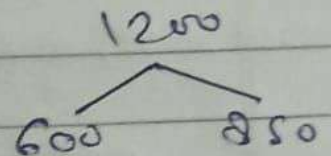
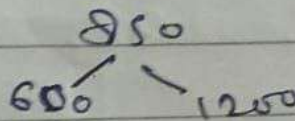


850, 1500, 1000, 600, 1200, 950

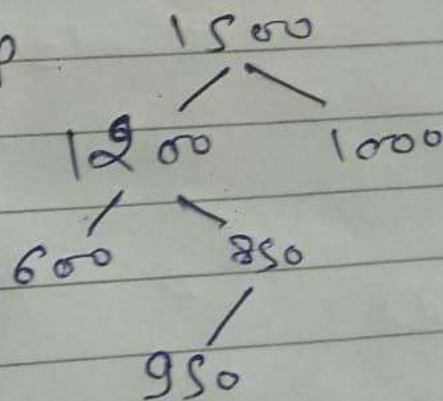
* Heapify Idx = 0



*



Max Heap



S-3

first Extn → swap root with

1500 ↔ 950

last element

950, 1200, 1000, 600, 850, 1500

Result

Q2

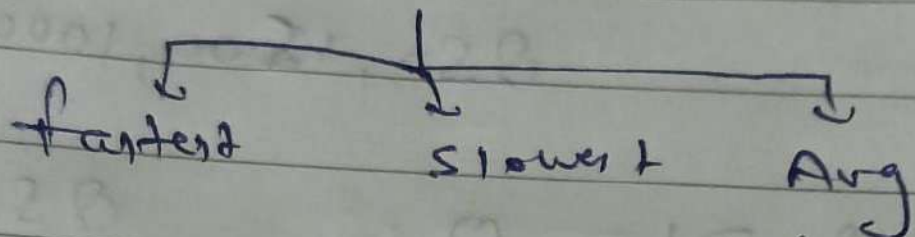
IP



Heap Sort



Sorted Arr



Count (< Avg)



Percentage